PROJETO INTERDISCIPLINAR

Requisitos da disciplina Modelagem de Software e Arquitetura de Sistemas

INTEGRANTES DO PROJETO e RA'S

Anna Paula Alves Silva	-	26029435
Dilly Martins da Silva	-	26029138
Laura Rayssa Souza Araujo	-	26029075
Rafaela Carvalho Barcos Mello	-	26029304
Matheus Gajewski de Melo	-	26029468

São Paulo

2026

Sumário

1 INTRODUÇÃO..... 3

1.1 – Processo de desenvolvimento de Software.....3

1.2 Apresentação.....3

2. DESIGN SPRINT – Ideação e prototipação do desafio..... 4

2.1 Desafio.....4

2.2 Entender Mapear..... 4

2.2.2 – Pesquisa Desk.....4

Gcompris: apresentação do jogo - descrição do jogo, pré-requisito (matéria utilizada exemplo, adição), objetivo (o que o aluno vai aprender), manual.....4

2.3 Ideação – desenho da solução (trilha do usuário).....5

2.4 Prototipagem.....5

3.REQUISITOS DE SISTEMA.....6

3.1 REQUISITOS FUNCIONAIS DE SOFTWARE..... 6

3.2 REQUISITOS NÃO FUNCIONAIS DE SOFTWARE.....9

4. CASOS DE USO..... 11

5. DIAGRAMA DE CLASSE..... 11

6. ARQUITETURA DO SISTEMA..... 11

6.1. Visão Geral do Sistema..... 11

6.2. Arquitetura Geral.....11

6.3. Componentes Arquiteturais e Camadas..... 11

6.4. Diagrama da Arquitetura..... 12

6.5. Organização de Pastas e Módulos..... 13

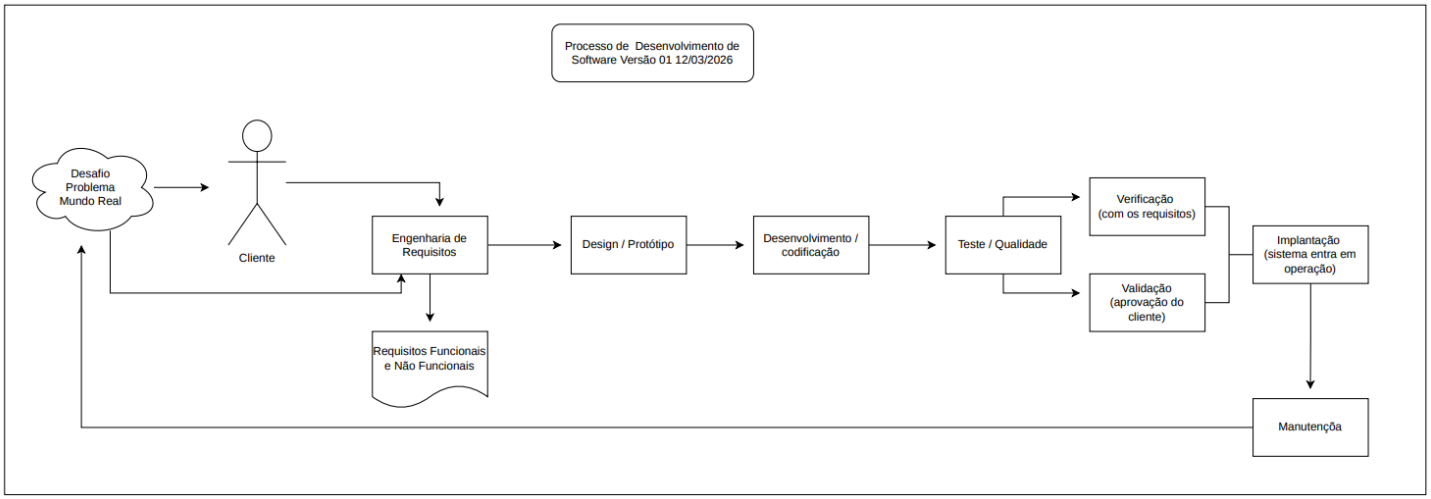
6.6. Integrações Externas (se houver)..... 13

6.7. Ambiente e Implantação..... 14

8. REFERÊNCIAS BIBLIOGRÁFICAS..... 14

1 INTRODUÇÃO

1.1 – Processo de desenvolvimento de Software



1.2 Apresentação

A Messier Data & Creative é uma empresa de desenvolvimento de jogos e experiências interativas que combina tecnologia e criatividade para promover engajamento em eventos, demonstrações de produtos e campanhas de marca. Como novo passo, a Messier pretende lançar uma plataforma de jogos digitais para escolas de ensino fundamental e médio. Neste Projeto Interdisciplinar, as equipes irão desenvolver um aplicativo Desktop para gerenciar assinaturas de jogos educacionais por escolas, controlando pacotes contratados, limites de acessos mensais, IPs autorizados e relatórios de consumo. Fluxo do processo (simulação do módulo Escola): 1) Login: a escola se autentica; o aplicativo verifica se a escola possui pacote ativo e se o IP de origem (informado na simulação) está autorizado. 2) Catálogo: o aplicativo apresenta os games disponíveis no pacote adquirido pela escola. 3) Acesso: a escola simula o acesso a um game; o aplicativo registra e contabiliza o acesso no consumo do mês, respeitando o limite do pacote.

2. DESIGN SPRINT – Ideação e prototipação do desafio

2.1 Desafio

Inicial da proposta do PI - As equipes irão desenvolver um aplicativo Desktop para gerenciar assinaturas de jogos educacionais por escolas, controlando pacotes contratados, limites de acessos mensais, IPs autorizados e relatórios de consumo.

2.2 Entender Mapear

2.2.1 – Entendimento e observação. COMO PODEMOS?

A segurança do sistema será baseada na validação de rede, onde o acesso é permitido exclusivamente através do endereço IP da instituição de ensino; caso o usuário tente se conectar de uma rede externa, a entrada será automaticamente negada. A verificação será realizada através da conferência de IP de rede, no qual o único autorizado sendo o da escola.

Para garantir a precisão dos dados de uso e diferenciar acessos reais de entradas acidentais, visto que existe um limite para quantos acessos são permitidos, o sistema implementará um rastreamento de fluxo via banco de dados: cada login iniciará uma contagem e somente após esse período de permanência ininterrupta, a sessão será oficialmente contabilizada e registrada no histórico do usuário.

2.2.2 – Pesquisa Desk

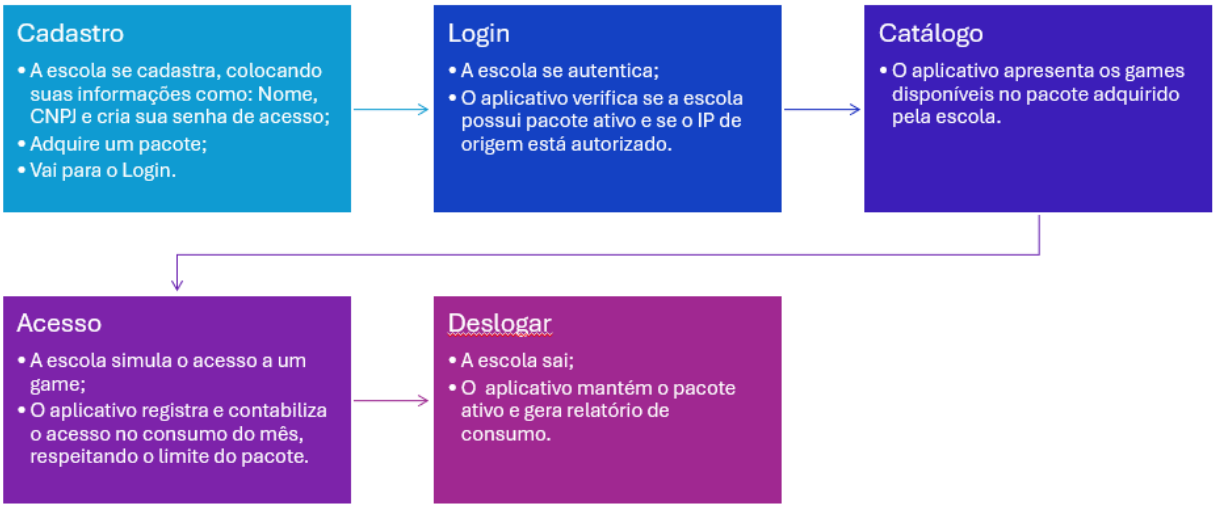
Gcompris: apresentação do jogo - descrição do jogo, pré-requisito (matéria utilizada exemplo, adição), objetivo (o que o aluno vai aprender), manual.

Escola games: carrossel dos jogos.

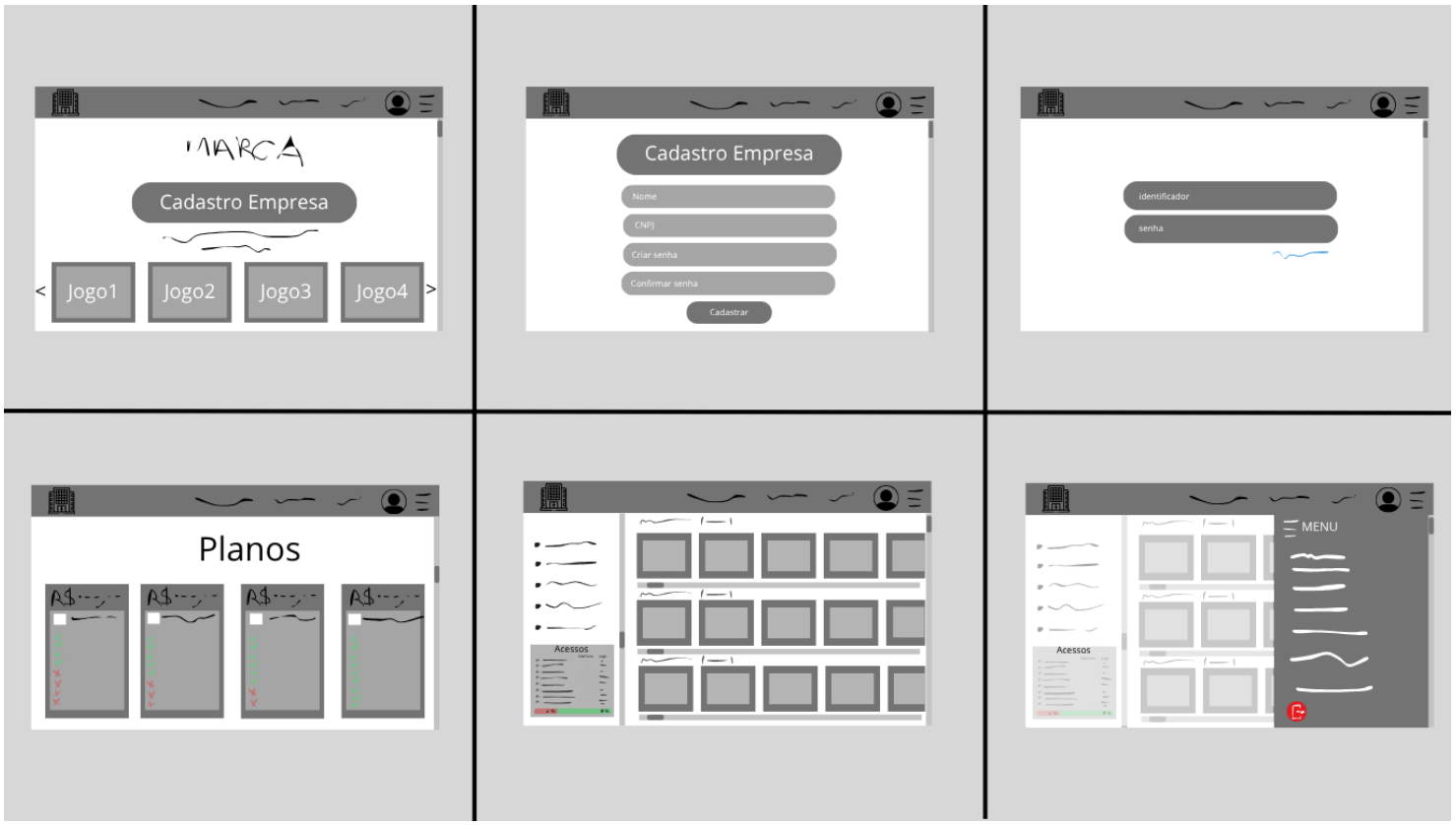
Nube: interface de login e informações diversas.

2.3 Ideação – desenho da solução (trilha do usuário)

Fluxo do processo (simulação do módulo Escola):



2.4 Prototipagem



3.REQUISITOS DE SISTEMA

3.1 REQUISITOS FUNCIONAIS DE SOFTWARE

RFS01	
Função	Cadastro_Games
Descrição	Cadastro de Games (CRUD) com nome, descrição, tema/disciplina, faixa etária e status ativo/inativo;
Entradas	
Fonte	
Saídas	
Ação	

RFS02	
Função	Cadastro_Pacotes
Descrição	Cadastro de Pacotes (CRUD) com: nome do pacote, lista de games incluídos, limite de acessos/mês e preço mensal;
Entradas	
Fonte	
Saídas	
Ação	

RFS03	
Função	Cadastro_Escolas
Descrição	Cadastro de Escolas (CRUD) com dados básicos, pacote adquirido (apenas um pacote ativo por escola) e lista de IPs autorizados;
Entradas	
Fonte	
Saídas	
Ação	

RFS04	
Função	Login_Escola
Descrição	Login do módulo Escola com validação de pacote ativo e verificação do IP de origem (informado na simulação);
Entradas	
Fonte	
Saídas	
Ação	

RFS05	
Função	Games_Disponiveis
Descrição	Exibição dos games disponíveis conforme o pacote ativo da escola;
Entradas	
Fonte	
Saídas	
Ação	

RFS06	
Função	Games_Registros
Descrição	Registro de acesso a game (log com data/hora, escola, game, IP, resultado permitido/bloqueado) e incremento do contador mensal;
Entradas	
Fonte	
Saídas	
Ação	

RFS07	
Função	Bloqueio_Acesso
Descrição	Bloqueio de acesso quando o limite mensal do pacote for atingido, com mensagem explicativa;
Entradas	
Fonte	
Saídas	
Ação	

RFS08

Função	Relatorio_Acesso
Descrição	Relatório de acessos por escola e por período (ex.: mês/ano), incluindo consumo acumulado;
Entradas	
Fonte	
Saídas	
Ação	

RFS09	
Função	Relatorio_Pacotes
Descrição	Relatório de escolas e pacotes (pacote vigente, limite mensal, consumo do mês e percentual de uso).
Entradas	
Fonte	
Saídas	
Ação	

3.2 REQUISITOS NÃO FUNCIONAIS DE SOFTWARE

RFS01	
Função	Aplicacao_Off
Descrição	A aplicação deve funcionar offline (modo local) para fins de protótipo;
Entradas	
Fonte	
Saídas	
Ação	

RFS02	
Função	Interface_Desktop
Descrição	Interface intuitiva e compatível com desktop (Windows), com telas de lista + formulário;
Entradas	
Fonte	
Saídas	
Ação	

RFS03	
Função	Tempo_Adequado
Descrição	Tempo de resposta adequado para consultas e relatórios com base de dados pequena/média;
Entradas	
Fonte	
Saídas	
Ação	

RFS04	
Função	Codigo_Camadas
Descrição	Código modular e organizado em camadas (UI, regras de negócio, dados/persistência);
Entradas	
Fonte	

Saídas	
Ação	

RFS05	
Função	Banco_Relacional
Descrição	Persistência em banco relacional com integridade (chaves, relacionamentos e validações);
Entradas	
Fonte	
Saídas	
Ação	

RFS06	
Função	Documento_Codigo
Descrição	Documentação clara e padronizada do projeto e do código (README e manual rápido de execução).
Entradas	
Fonte	
Saídas	
Ação	

4. CASOS DE USO

Apresentar 3 casos de uso do sistema

5. DIAGRAMA DE CLASSE

6. ARQUITETURA DO SISTEMA

Para a arquitetura do sistema, considere o template abaixo:

Template (esquema de arquitetura) para orientar a elaboração de um **Projeto de Arquitetura de Sistema Web**, cobrindo desde a camada de apresentação até a persistência de dados, com foco em modularidade, escalabilidade e eficiência.

6.1. Visão Geral do Sistema

- **Nome do Sistema**:
- **Objetivo do Sistema**: (Breve descrição da finalidade do sistema)
- **Público-Alvo**:
- **Principais Funcionalidades**:

6.2. Arquitetura Geral

Descrever o estilo arquitetural adotado (por exemplo, MVC, camadas, microsserviços, hexagonal, etc.):

Exemplo: Arquitetura em Camadas (Layered Architecture) com separação clara entre apresentação, lógica de negócios e persistência.

6.3. Componentes Arquiteturais e Camadas

6.3.1. Camada de Apresentação (Frontend)

- **Tecnologias**: (Ex.: HTML5, CSS3, JavaScript, React/Vue/Angular)
- **Responsabilidades**:
    - Interface com o usuário
    - Validação inicial de dados
    - Comunicação com a API via HTTP/JSON

6.3.2. Camada de Aplicação (Backend/API)

- **Tecnologias**: (Ex.: Node.js, Django, Spring Boot, Flask, Laravel)
- **Responsabilidades**:
  - Implementação da lógica de negócio
  - Autenticação/autorização
  - Comunicação com a camada de dados
  - Exposição de endpoints RESTful/GraphQL

6.3.3. Camada de Domínio (Negócio)

- **Descrição**:
  - Contém as regras de negócio e os modelos de domínio
  - Independente das demais camadas
- **Componentes**:
  - Entidades
  - Casos de uso
  - Regras de validação

6.3.4. Camada de Persistência (Banco de Dados)

- **Tecnologias**: (Ex.: PostgreSQL, MySQL, MongoDB, SQLite)
- **Responsabilidades**:
  - Armazenamento e recuperação de dados
  - Scripts de criação e migração de banco

6.4. Diagrama da Arquitetura

Inserir um **diagrama estrutural** (ex.: em draw.io, Lucidchart ou outro) mostrando:

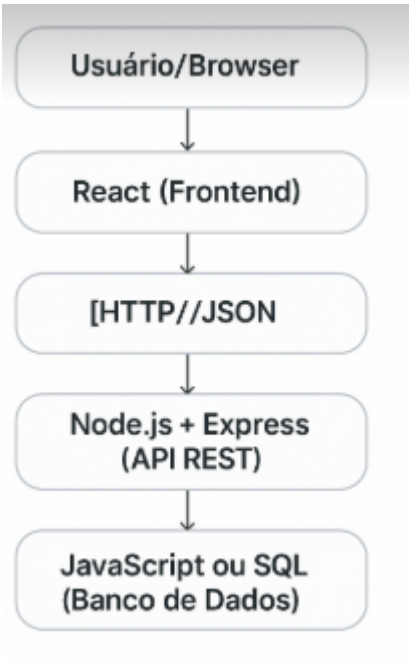
- Fluxo entre frontend, backend, banco de dados
- Componentes principais
- APIs e serviços envolvidos
- Gateways, autenticação, filas de mensagens (se houver)

Exemplo textual de uma arquitetura de sistema web.

[Usuário/Browser]

|
React (Frontend)
|
[HTTP/JSON]
|
Node.js + Express (API REST)
|
JavaScript ou SQL
|
PostgreSQL (Banco de Dados)

Diagrama:



6.5. Organização de Pastas e Módulos

Descrever como o código será organizado no repositório:

```
📁 /frontend
    📁 components/
    📁 pages/
    📁 services/
📁 /backend
    📁 controllers/
    📁 services/
    📁 models/
    📁 routes/
    📁 config/
📁 /database
    📄 schema.sql
    📄 migrations/
```

6.6. Integrações Externas (se houver)

- APIs de terceiros (pagamentos, mapas, redes sociais)

- Serviços de autenticação (OAuth2, Firebase, etc.)
- Serviços de envio (email, notificações)

6.7. Ambiente e Implantação

- **Ambientes**: desenvolvimento, homologação, produção
- **Contêineres**: Docker/Docker Compose (se usado)
- **CI/CD**: GitHub Actions, GitLab CI, Jenkins
- **Hospedagem**: Heroku, Vercel, AWS, GCP, Azure

8. REFERÊNCIAS BIBLIOGRÁFICAS

SOMMERVILLE, I. **Engenharia de Software.** 11ª Edição. São Paulo: Pearson Addison-Wesley, 2017.